

Декларативный списочный метод (Declarative list)

Декларативный списочный метод (англ. Declarative list) — это [списочный метод БД](#), который позволяет в удобной форме создавать запросы к БД для получения выборки по заданным условиям, но при этом не требующий знаний языков программирования.

Для работы декларативного метода необходимо, чтобы в состав сервиса приложения был подключен сервисный модуль "Бизнес логика", который поставляется в составе [SBIS SDK](#).

Входные параметры

ДопПоля (ExtraFields)

Набор полей, которыми будут расширены возвращаемые записи.

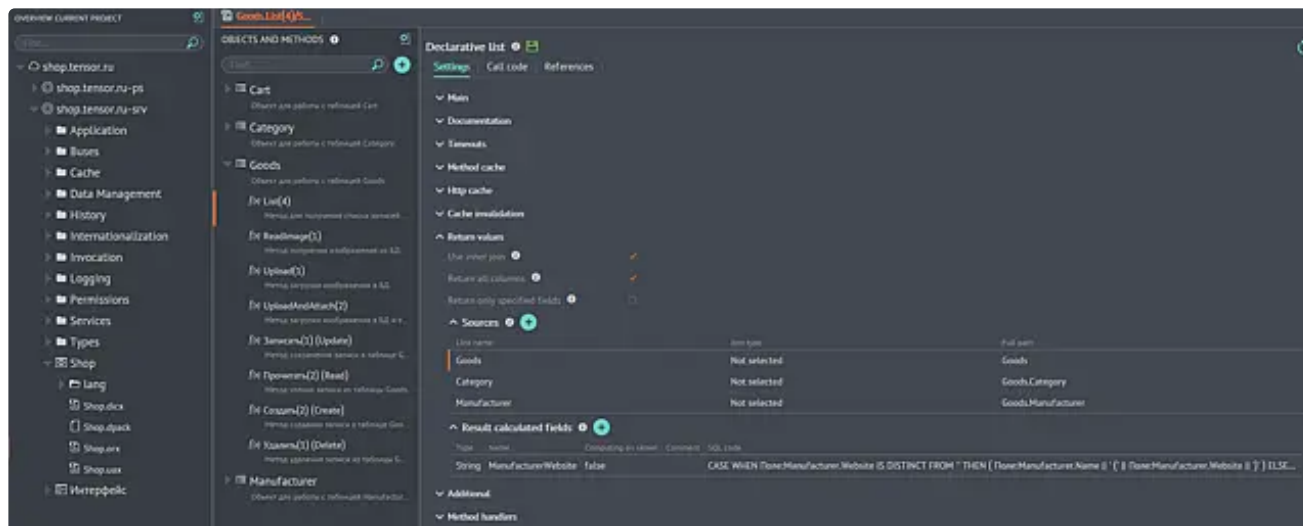
В опции [Result source fields](#) задаётся набор полей, из которых формируются возвращаемые записи. В таком наборе могут присутствовать только поля, которые изначально определены в таблицах БД, на работу с которым настроен декларативный метод (см. [Sources](#)).

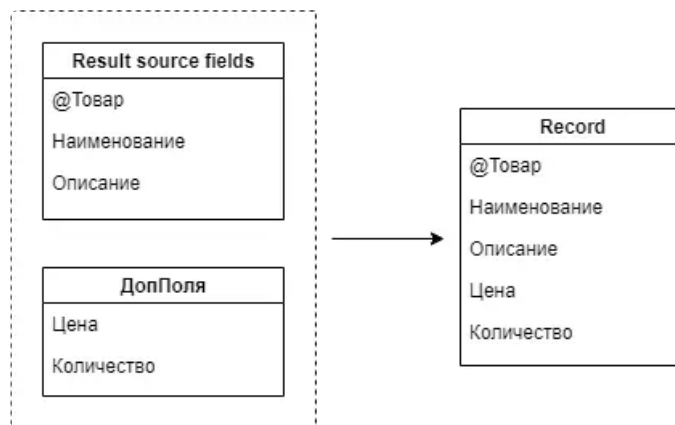
Если таблица БД состоит из множества полей, а в выборке нужно получить лишь часть, то разработчик вправе по умолчанию задать ограниченный список полей в опции **Result source fields**. Таким образом метод будет выполнен быстрее.

В сценариях, где набор возвращаемых полей требуется расширить, следует использовать входной параметр **ДопПоля**. Благодаря такому подходу исключается дублирование методов, отличающихся только набором возвращаемых полей.

Передача параметра **ДопПоля** не имеет смысла, когда в опции Return all columns установлено значение true.

Рассмотрим пример. В опции **Result source fields** установлено три поля: "@Товар", "Наименование" и "Описание". В параметре **ДопПоля** установлено два поля: "Цена" и "Количество". В итоге записи выборки будут состоять из 5 полей: @Товар, "Наименование", "Описание", "Цена" и "Количество".





Фильтр (Filter)

Параметры фильтрации выборки.

Значения параметров могут быть использованы при создании [условий фильтрации](#). Без использования в условиях фильтрации никак не влияют на результаты выборки.

Сортировка (Sorting)

Параметры сортировки записей в выборке.

Выборка может быть отсортирована по нескольким полям. Сортировка выполняется последовательно: от первого до последнего параметра из набора. Значения входного параметра "Сортировка" имеют приоритет над сортировкой, которая устанавливается в опции [Sorting parameters](#).

Сортировка на стороне бизнес-логики представлена классом [SortingList](#), который является массивом [ColumnSortParameters](#).

Для конфигурации параметра сортировки по полю применяют следующие опции:

- **fieldName** — имя поля, по которому производится сортировка.
- **order** — порядок сортировки. Возможные значения:
 - `sbis.SortOrder.soDESCENDING` — обратный порядок сортировки (по убыванию, DESC);
 - `sbis.SortOrder.soASCENDING` — прямой порядок сортировки (по возрастанию/ алфавитный порядок, ASC).
- **nullPolicy** — политика обращения с NULL-значениями. Возможные значения:
 - `true` — записи расположены в конце сортируемой последовательности;
 - `false` — записи с NULL-значениями расположены в начале сортируемой последовательности.

Навигация (Pagination)

Параметры для навигации с типом [постраничная](#) или [по курсору](#).

Навигация на стороне бизнес-логики представлена классом [Navigation](#).

Информацию о конструкторах для навигации можно посмотреть [здесь](#).

Опции:

- **Limit** — число записей на странице.
- **Page** — номер страницы, записи которой будут возвращены декларативным методом. Нумерация страниц начинается с нуля.
- **HasMore** — признак наличия следующих страниц выборки. Возможные значения:
 - `true` — в ответе от сервера бизнес-логики будет находиться признак: есть следующие страницы.
 - `false` — в ответе от сервера бизнес-логики возвращается общее количество записей выборки.

Если данный параметр не был передан, то сервер бизнес-логики не возвращает никаких дополнительных данных об общем количестве записей.
- **Position** — курсор. Применяется для списков с [навигацией по курсору](#).
- **Direction** — направление выборки относительно курсора. Возможные значения:
 - `bothways`,
 - `forward`,
 - `backward`.

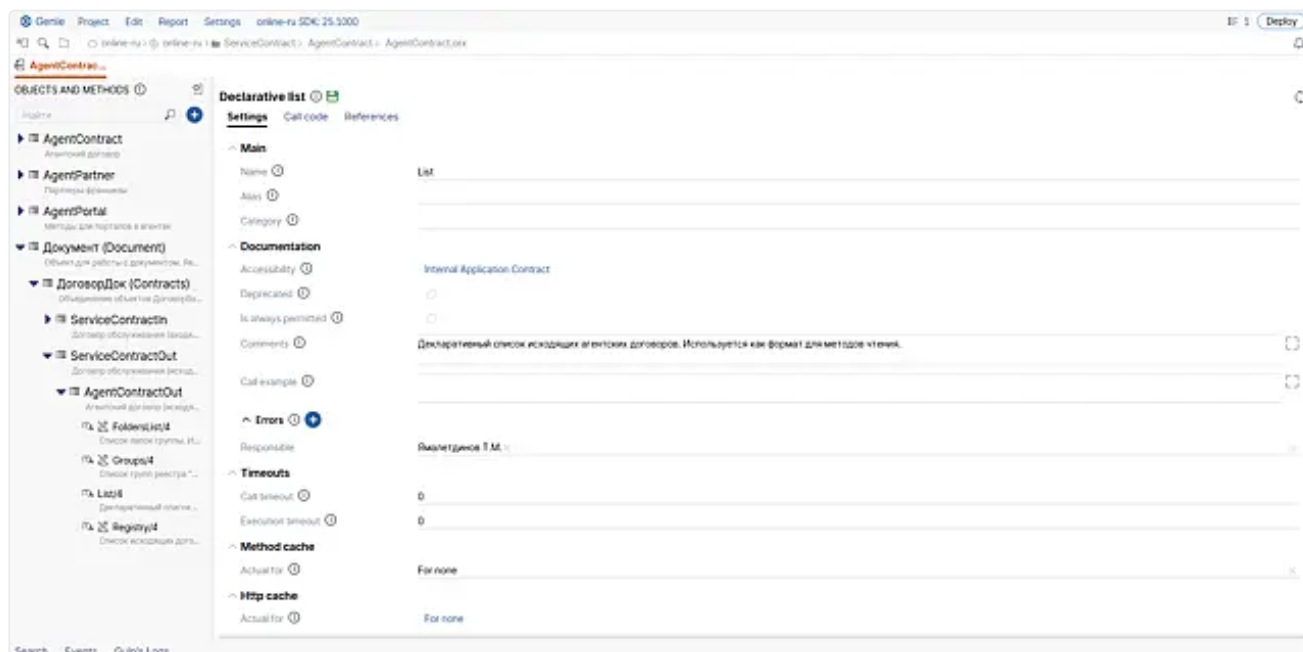


Декларативный метод запрашивает из БД на одну запись больше, чтобы можно было ответить на вопрос "есть ли еще".

Для постраничной навигации эта лишняя запись отрезается на этапе сериализации. При навигации по курсору в декларативном списке нужно отрезать лишнюю запись самостоятельно внутри списка.

Атрибуты, настраиваемые через интерфейс Genie

Руководство по созданию декларативного списочного метода в [Genie](#) описано [здесь](#).



Name

Название метода.

В названии допускается использование букв латинского алфавита, цифр, символа подчеркивания и дефиса. Оно должно быть уникальным внутри одного объекта и лаконичным. Сейчас допустимо использование русскоязычных названий, но в дальнейшем они поддерживаться не будут.

Имя должно отражать предназначение метода. Например, если назвать "СписокТоваров", то всем станет понятно, что метод нужен для получения списка товаров. Название из нескольких слов следует писать слитно в стиле [CamelCase](#), например, GetPageFiles.

Alias

Псевдоним метода.

Является альтернативным названием, по которому метод БЛ доступен для вызова. Псевдонимы, как правило, служат для обеспечения совместимости на время для перехода от русскоязычных названий к англоязычным.

Чтобы метод вызвать по псевдониму, нужно использовать псевдоним объекта БЛ. Если вы создаёте новый метод и задали англоязычное название, то проигнорируйте данную опцию (оставьте её пустой). Если вы редактируете уже созданный метод, то настоятельно рекомендуется задать псевдоним согласно правилам именования, которые описаны в атрибуте [Name](#).

Метод можно вызвать двумя способами:

- ИмяОбъекта.ИмяМетода
- ПсевдонимОбъекта.ПсевдонимМетода

Для вызова метода по псевдониму необходимо его наличие и у метода и у объекта.

Category

Категория для группировки методов в рамках одного объекта. Отображается как директория. Значение - произвольная строка.

Accessibility

Подробное описание атрибута читайте [здесь](#).

Deprecated

Признак информирует разработчиков об устаревшем методе, который не следует использовать.

Обязательно укажите в поле **Deprecation reason** (отображается при отметке флага **Deprecated**) причину устаревания метода, например, более корректный аналог.

Методы считаются устаревшими, если изменяется его название или сигнатура. Устаревшие методы в дальнейшем поддерживаться не будут, и в течение нескольких последующих версий они будут удалены.

При вызове устаревшего метода (уровня Service contract и выше) в логах ошибок сервиса будут добавлена соответствующая информация, по которой администратор системы должен предпринять соответствующие работы по замене используемых методов.

Is always permitted

Признак информирует разработчиков о методе, права на который не проверяются при его вызове.

Обязательно укажите в поле **Always permitted reason** (отображается при отметке флага **Is always permitted**) причину отсутствия проверки прав на метод.

Comment

Пользовательский комментарий. Создание комментариев считается правилом хорошего тона, особенно если вы работаете в команде.

Содержательные и максимально краткие комментарии помогают другим разработчикам быстрее понять предназначение метода и узнать некоторые детали работы с ним.

Return values

Тип значения, которое возвращает метод БЛ. Из исходного кода метода разработчик должен вернуть именно такой тип.

Use inner join

Тип объединения полей записи при их выборке из связанных таблиц, указанных источниками текущего метода. Inner Join - оператор внутреннего соединения, порядок таблиц неважен, поскольку оператор является симметричным.

Объединение таблиц Join подробно описано в [документации PostgreSQL](#).

Return only specified fields

Признак, по которому декларативный список может возвращать только поля, указанные в атрибуте [Result source fields](#).

При установленном флаге входной параметр [ДопПоля \(ExtraFields\)](#) не обрабатывается, а метод **Прочитать** по этому списку возвращает только **Result source fields**, а не все поля, предоставляемые источником данных.



По умолчанию метод "Прочитать" возвращает все поля, содержащиеся в источнике данных (если таблица - все поля таблицы, если метод - все поля возвращаемые методом).

Пример:

- В базовом методе "Объект1.БазовыйМетод" указан источник данных таблица, установлен флаг **Return only specified fields**, указаны возвращаемые поля.
- В производном методе "Объект2.ПроизводныйМетод", указан источник "базовый метод", установлен или не установлен флаг **Return only specified fields**, указаны возвращаемые поля (источника данных (базовый метод) + поля из таблицы).

Ожидаемое поведение:

Т.к. источник данных производного метода - декларативный метод, то полями для метода "Прочитать" будут все поля базового метода, а при установленном флаге **Return only specified fields** - только ограниченный набор полей, т.е. производный метод оперирует только тем, что вернул базовый метод.

Если необходимо расширение состава полей в производном методе, установите флаг **Return only specified fields** только в производном методе, а в базовом - снимите отметку. В этом случае базовый метод при вызове "Прочитать" будет возвращать все поля источника данных (таблицы), а не только те, которые указаны в **Result source fields**.

Как результат, производный метод будет оперировать всеми полями таблицы и может ограничить состав возвращаемых полей установкой флага **Return only specified fields**.

Sources

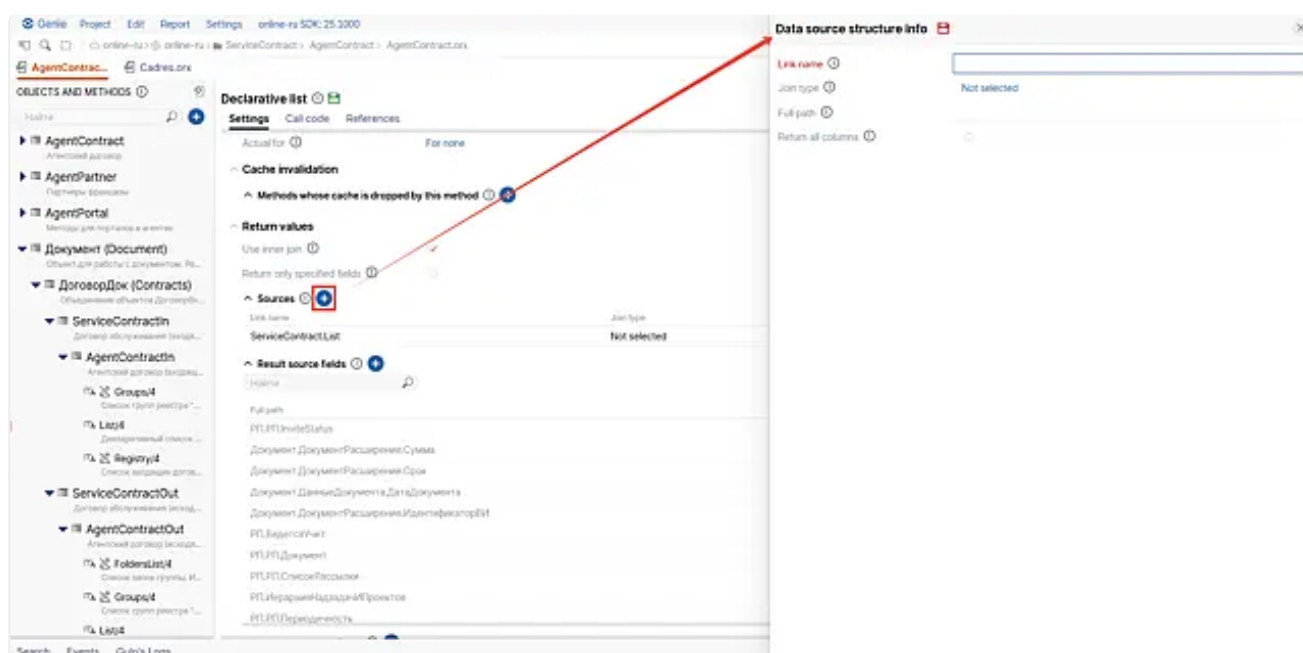
В этом атрибуте указывают источники данных для декларативного метода — таблицы БД или декларативные списочные методы, к которым обращается метод для получения выборки.

Если первым источником выбрана таблица БД, то дальнейший выбор разрешён только среди таблиц, которые связаны с первым источником любым типом [отношений](#).

Список источников, которые можно выбрать в атрибуте из интерфейса Genie, формируется из словарей данных и справочников объектов, существующих в рамках общего сервиса приложения. Когда источники не заданы, конфигурация метода считается невалидной.

Клик по кнопке "+" открывает окно **Data source structure info**, содержащее атрибуты:

- **Link name** - имя связи;
- **Join type** - тип связи;
- **Full path** - полный путь;
- **Return all columns** - вернуть все колонки. Если установлен этот флаг, выборка декларативного списочного метода будет создана из всех полей, доступных для выбора в атрибуте [Result source fields](#).



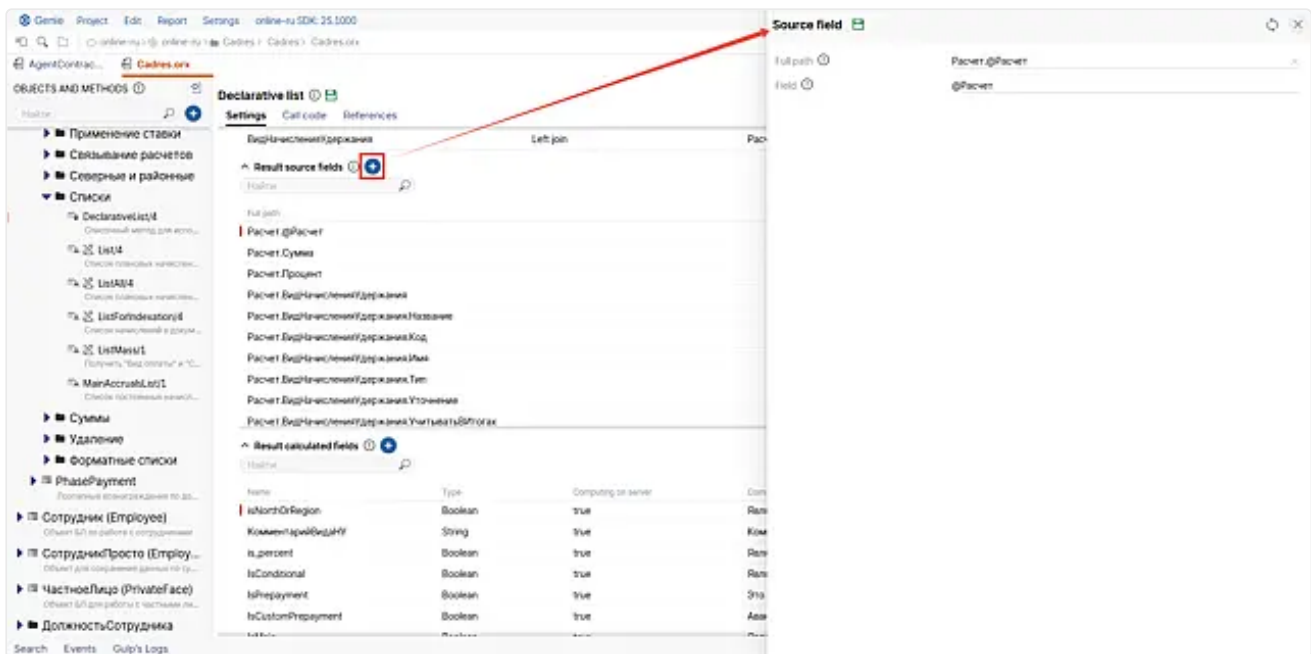
Result source fields

В этой опции выбирают поля, по которым декларативный списочный метод создаст выборку данных. Список полей определяется теми источникам данных, которые указаны в атрибуте [Sources](#). Чтобы выбрать все поля, установите флаг **Return all columns**.

Первым полем обязательно должен возвращаться уникальный целочисленный идентификатор. Как правило, это первичный ключ из базовой таблицы объекта.

Клик по кнопке "+" открывает окно **Source field**, содержащее атрибуты:

- **Full path** - полный путь;
- **Field** - имя поля таблицы.



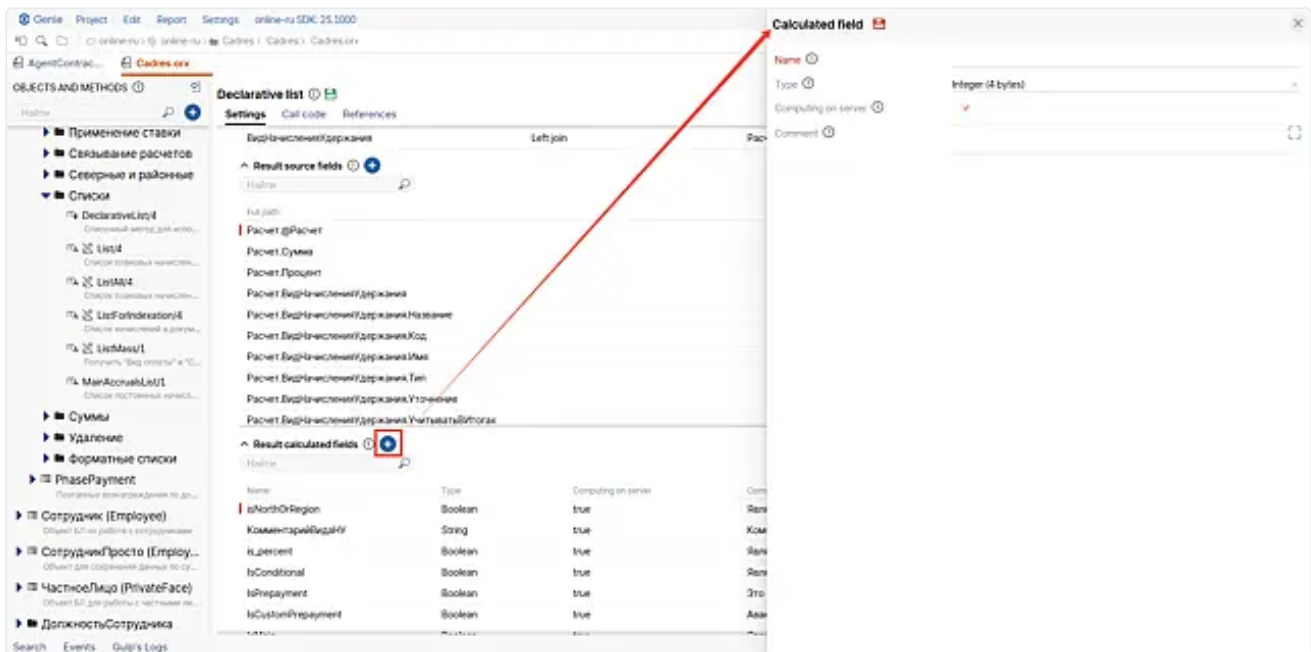
Result calculated fields

В этой опции создают рассчитываемые поля. Они вычисляются на сервисе приложения в ходе выполнения декларативного метода и включаются в состав результирующей выборки. Значения рассчитываемых полей не хранятся в БД.

Рассчитываемые поля используются там, где нужно расширить формат записей, возвращаемых методом БЛ, без изменения структуры БД.

Клик по кнопке "+" открывает окно **Calculated field**, содержащее атрибуты:

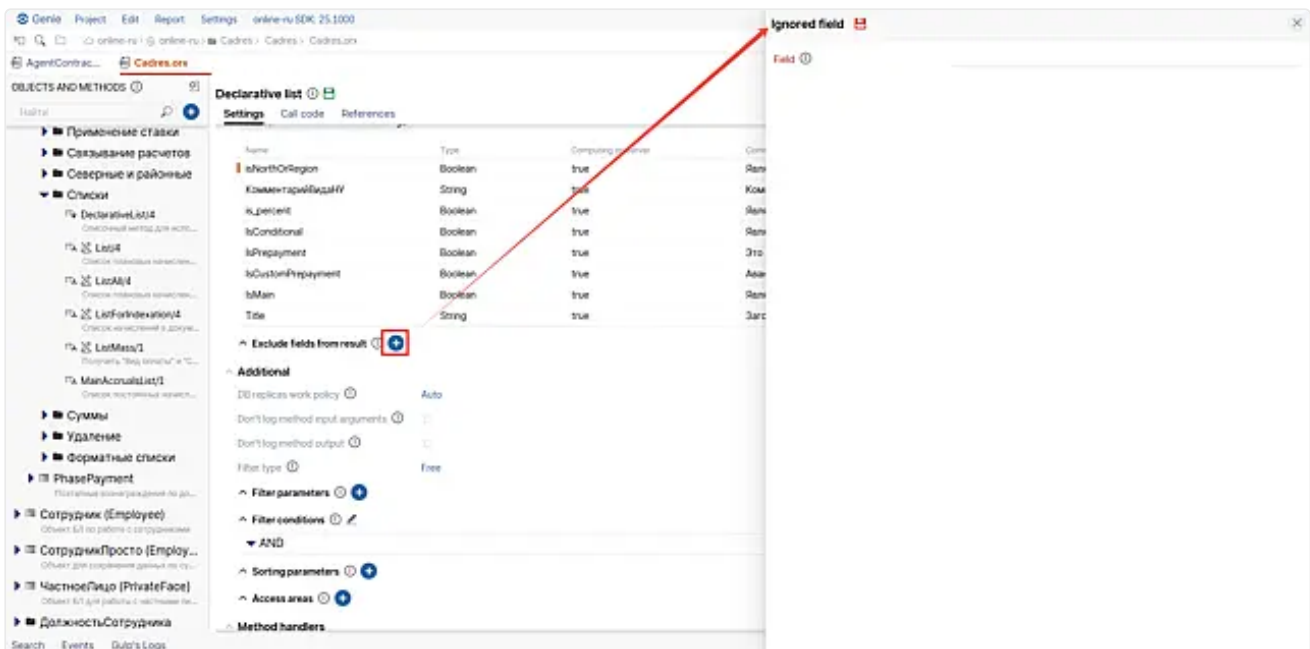
- **Name** - название рассчитываемого поля;
- **Type** - тип поля;
- **Computing on server** - признак, по которому поле будет рассчитываться при помощи обработчика на Python или C++;
- **Comment** - пользовательский комментарий.



Exclude fields from result

Список исключаемых полей, по которым не будет произведена выборка данных. Имеет смысл для унаследованных полей.

Клик по кнопке "+" открывает окно **Ignored field**, в котором в атрибуте **Field** указывается исключаемое из результата поле.



Don't log method input arguments

Установите отметку на этом флаге, если метод принимает в качестве аргументов конфиденциальные данные.

Стандартный сценарий использования методов бизнес-логики подразумевает логирование их вызова с указанием аргументов. Однако методы могут работать с конфиденциальными данными, которые передаются как его аргументы. Например, методы сервиса "Аутентификация" могут использовать логины, пароли и адреса электронной почты пользователей.

В целях безопасности следует установить флаг, чтобы отключить логирование аргументов метода, и предотвратить случайную утечку конфиденциальной информации.

Don't log method output

Установите отметку на этом флаге, если метод возвращает в качестве результата конфиденциальные данные.

В типичном сценарии результат RPC-вызова логируется и доступен для анализа в облачной консоли [Журнала логов](#). Однако методы бизнес-логики могут работать с конфиденциальными данными и выдавать их в качестве результата. Например, это могут быть паспортные данные, которые в определённых сценариях требуется передать в браузер.

В целях безопасности следует установить флаг, чтобы отключить логирование результата метода, и предотвратить случайную утечку конфиденциальной информации.

Frequent call protection

Включает функционал, контролирующий [частоту вызова метода бизнес-логики](#).

Filter type

[Тип фильтра](#). Определяет доступный набор параметров фильтрации.

Filter parameters

Параметры фильтрации. Применяются при создании [условий фильтрации](#).

Filter conditions

Условия фильтрации, по которым будет производиться отбор возвращаемых записей. Метод генерирует SQL-код с условиями из списка [WHERE](#).

В поле **Predicate** задается тип выражения для условия фильтрации:

- Simple Predicate - простое выражение, например, «Значение поля X больше значения параметра фильтра Y»;
- SQL expression predicate - сложное SQL-выражение. При добавлении SQL-выражений доступ к параметру фильтрации задается в формате **Фильтр:ИмяПараметра**, а доступ к полю записи — **Поле:НазваниеПоля**.
- Composite predicate - составное выражение.

В качестве параметров выражений можно использовать поля структуры метода ([Sources](#)) и параметры фильтрации ([Filter parameters](#)).

Простое условие

Простое выражение в условии фильтрации — это логическое утверждение, состоящее из одного оператора сравнения (равно, больше, меньше) между полем данных и значением.

Для добавления условия предназначены следующие параметры:

- **Left type** и **Right type** - типы операндов.

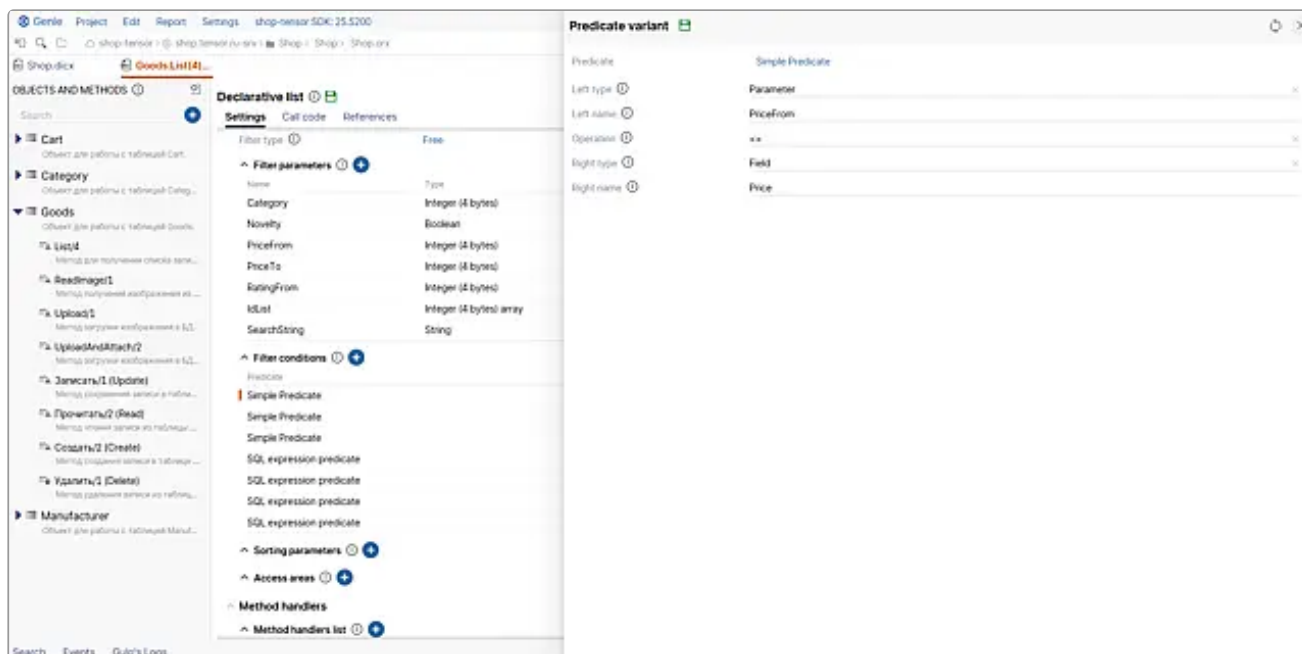
Допустимые значения:

- Scalar - скалярное значение.
- Field - поле фильтра.
- Parameter - параметр фильтра.
- Null value - нулевое значение.
- Placeholder - прототип.
- Object parameter - устаревшее значение (не рекомендуется к использованию).
- Function - название хранимой процедуры.

- **Left name** и **Right name** - имена операндов.

- **Operation** - применяемый оператор SQL.

- == - равно
- > - больше
- < - меньше
- >= - больше или равно
- <= - меньше или равно
- != - не равно
- is - является
- is not - не является



Сложное SQL-выражение

Условие фильтрации задается в виде SQL-кода. При добавлении SQL-выражений доступ к параметру фильтрации задается в формате **Фильтр:ИмяПараметра**, а доступ к полю записи — **Поле:НазваниеПоля**.

Сортировка используется для удобного представления результатов выбора записей из БД. В пользовательском интерфейсе сортировка может пригодиться, например, для расположения списка товаров в порядке возрастания

цены. В этом случае сортировка улучшает поведенческий фактор в вашем веб-приложении, а если речь идёт об интернет-магазине, то методы с сортировкой результатов помогут быстрее определиться с выбором покупки.

Access areas

[Область видимости](#) метода.

Method handlers

В этой опции можно задать обработчики, которые запускаются сразу после выполнения декларативного списочного метода. Код обработчиков можно реализовать на Python или C++, о чем подробнее читайте в [статье](#).

Enable cache

Включает [серверное кэширование](#) результатов выполнения метода.

Кэшируются данные для кода, выполняющегося в рамках одного RPC-вызова.

- **Life time** - время жизни кэша. Период времени, в течение которого будут храниться кэшированные данные. Параметр может принимать значения от 1 мин до 48 часов. Выбор периода хранения данных должен соответствовать частоте вызова метода с различными значениями аргументов. Если частоту вызова предугадать трудно, то установите максимальное время жизни кэша.

- **Actual for** - область актуальности кэшированных данных, которая определяет для кого они будут доступны.

Параметр принимает следующие значения:

- Для сессии - кэширует данные для пользователя в пределах его сессии. Например, подходит для методов фильтрации по заданным параметрам в рамках одного списка.
- Для пользователя - кэширует данные для пользователя независимо от его сессии.
- Для клиента - кэширует данные для всех пользователей клиента. Например, подходит для методов, возвращающих контент веб-приложения, соответствующий определённой группе пользователей.
- Для всех - кэширует данные для всех клиентов и их пользователей. Например, подходит для методов, отображающих общие интерфейсные элементы навигации.

Enable http cache

Включает клиентское кэширование данных (в веб-браузера). Используется для статической информации, которая изменяется очень редко, либо происходит изменение ссылки, по которой она доступна.

Для примера, браузерное кэширование используется на online.sbis.ru в сервисе новостей для хранения картинок.

Применяется только для Get-запросов. В ответе на Get-запрос добавляются заголовки Cache-Control и Vary, которыми задаётся время жизни кэша и его актуальность.

Для браузерного кэширования не применяется инвалидация кэша. В качестве её аналога можно использовать изменение ссылки на данные (например, ссылка на картинку).

Значения подопций **Life time** и **Actual for** аналогично предыдущему пункту.

Расчет строки итогов с помощью декларативного метода

Декларативный списочный метод поддерживает автоматический расчет [строки итогов](#) по [числовым полям](#).

Чтобы метод рассчитал строку итогов, при вызове метода в входном параметре [Фильтр \(Filter\)](#) передайте параметр с именем **_Итоги**. Значением такого параметра является запись со следующими полями:

- [ПоляРасчета](#). Массив названий числовых полей таблицы, по которым производить суммирование значений для строки итогов.
- [ВидРасчета](#). Признак: по каким типам записей производить расчет: "По листьям", "По узлам" или "По узлам и листьям". Подробнее о различиях между каждым типом записей вы можете прочитать в разделе [Типы отношений в таблицах БД](#).

Расчет значения поля Раздел\$ с помощью параметра _ЕстьДочерние

В [реестрах](#) при перемещении записи пустые разделы должны отображаться без стрелочки, с чем прикладному разработчику должно помочь поле Раздел\$ (true — раздел не пуст, false — раздел пуст).

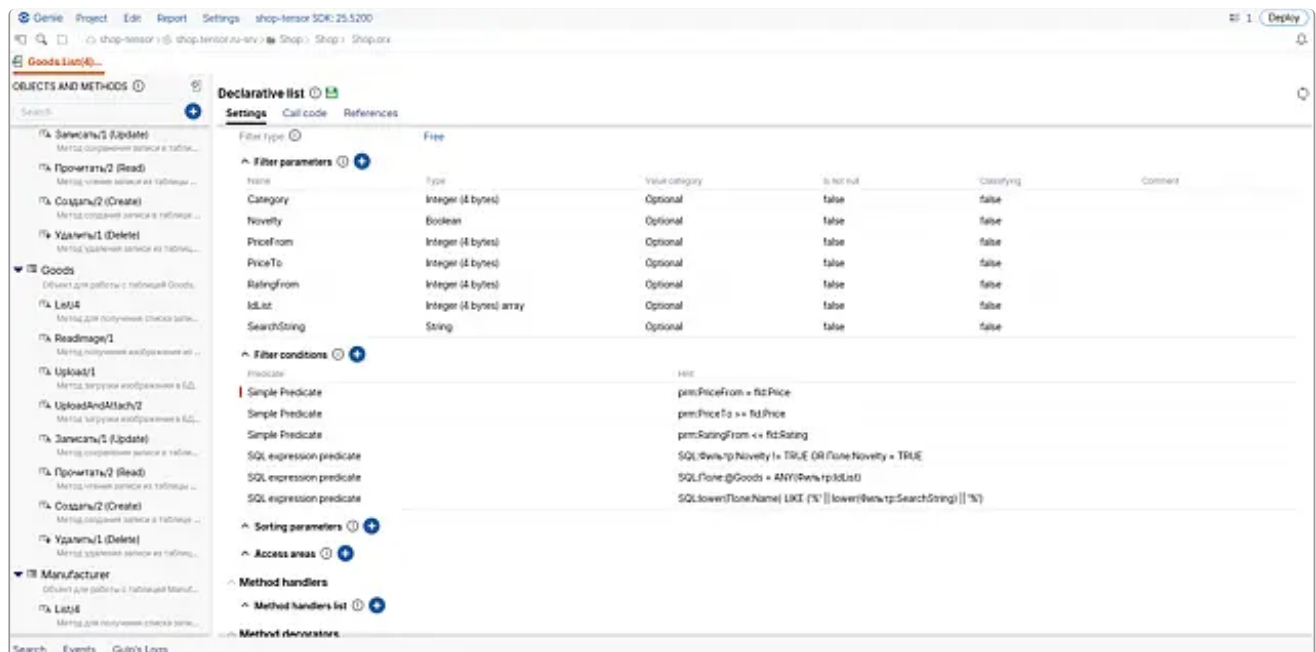
Вернуть значение поможет параметр "_ЕстьДочерние":true, который нужно передать в параметрах фильтрации декларативного списочного метода.

Построение хлебных крошек при помощи параметра "ПутьКУзлу"

При необходимости построения [хлебных крошек](#) необходимо передать в составе параметра "Фильтр" поле ПутьКУзлу со значением true.

Примеры

Пример 1. Конфигурация декларативного метода в Genie



При такой конфигурации:

- выборка производится из таблицы "Товар";
- возвращаемые записи состоят из трех полей "@Товар" (ПК), "Наименование" (текстовое поле) и "Описание" (текстовое поле).
- фильтрация записей по категории "Ноутбуки";
- сортировка записей по полю "Цена" в порядке возрастания.

По данным параметрам будет сформирован следующий SQL-запрос к БД:

```
SELECT "@Товар", "Наименование", "Описание" FROM "Товар" WHERE  
"Категория"="Ноутбуки" ORDER BY "Цена" ASC
```

Пример 2. Вызов декларативного метода из исходного кода

JavaScript

```
1 import {SbisService, Query, QueryOrder} from 'Types/source';  
2  
3 const query = new Query()  
4
```

```

5 // параметр "ДопПоля"
6 .select(['Производитель', 'Брутто'])
7
8 // параметр "Фильтр"
9 .where({
10     Новика: true,
11     Рейтинг: 5
12 })
13
14 // параметр "Сортировка"
15 .orderBy('Цена', QueryOrder.SORT_DESC, QueryOrder.NULL_POLICY_LAST);
16
17 const source = new SbisService({
18     endpoint: 'Склад',
19     binding: {query: 'СписокТоваров'}
20 });
21
22 // вызов декларативного метода
23 source.query(query).then((response) => {
24     const goodsList = response.getAll();
25 });

```

Используемые классы и методы: [BLObject](#) и [query](#). При вызове не передается параметр "ДопПоля".

Python

```

1 # параметр "ДопПоля"
2 fields = ['Производитель', 'Брутто']
3
4 # параметр "Фильтр"
5 filter_rec = sbis.Record()
6 filter_rec.AddBool('Новика', true)
7 filter_rec.AddInt16('Рейтинг', 5)
8
9 # параметр "Сортировка"
10 sorting = sbis.SortingList()
11 sort_param_price = sbis.ColumnSortParameters()
12 sort_param_price.fieldName = 'Цена'
13 sort_param_price.order = sbis.SortOrder.soDESCENDING
14 sort_param_price.nullPolicy = sbis.NullSortPolicy.nspNULLS_LAST
15 sorting.push_back(sort_param_price)
16
17 # вызов декларативного метода
18 rs = sbis.Goods.List(fields, filter_rec, sorting, None)

```

Используемые классы и методы: [Record](#), [AddBool](#), [AddInt16](#), [AddString](#), [Navigation](#), [SortingList](#), [ColumnSortParameters](#), [SortOrder](#), [NullSortPolicy](#) и [push_back](#).

C++

```

1 // параметр "ДопПоля"
2 dbi::StringList fields;
3 fields.push_back( L"Производитель" );
4 fields.push_back( L"Брутто" );
5

```

```
6 // параметр "Фильтр"
7 Record filter;
8 filter.AddBool( L"Новика", true );
9 filter.AddInt16( L"Рейтинг", 5 );
10
11 // параметр "Сортировка"
12 dbi::SortingList sorting;
13 sorting.push_back( ColumnSortParameters( L"Цена", dbi::soDESCENDING,
14 dbi::nspNULLS_LAST ) );
15
16 // вызов декларативного метода
17 bl::Object( L"Склад" ).Invoke<RecordSet>( L"СписокТоваров", fields,
18 filter, sorting, NULL );
```

Используемые классы и методы: [Record](#), [Navigation](#) и [Object](#).